

Chapter 47 — Rolling Hash / Rabin-Karp

0. What You Will Learn in This Chapter

Rolling Hash is an advanced string technique used to compare substrings efficiently. Rabin-Karp is a string matching algorithm that uses rolling hash to find a pattern inside text.

Rolling hash is useful when you need to:

- compare many substrings quickly
- find duplicate substrings
- find repeated sequences
- count distinct substrings
- search patterns
- combine binary search with substring checking
- check palindromes with forward/reverse hash
- solve string problems where KMP is not flexible enough
- avoid $O(n^2 * \text{length})$ substring comparisons

By the end of this chapter, you should understand:

- What hashing means for strings
- What polynomial rolling hash is
- How to compute prefix hash
- How to get hash of any substring in $O(1)$
- What Rabin-Karp is
- How rolling hash supports binary search
- Why collisions can happen
- How double hashing reduces collision risk
- How to solve longest duplicate substring
- How to solve repeated DNA sequences
- How to count distinct substrings
- When to use KMP, Trie, or rolling hash

This is a hard string chapter, so it includes extra explanations and examples.

Core problems covered in this chapter:

1. Rabin-Karp Pattern Matching
 2. Repeated DNA Sequences
 3. Longest Duplicate Substring
 4. Distinct Echo Substrings
 5. Count Distinct Substrings
 6. Longest Happy Prefix with hash
 7. Shortest Palindrome with hash
 8. Palindrome checking with forward/reverse hash
 9. String matching with multiple patterns, preview
 10. Double hashing
 11. Collision handling
 12. Rolling hash for integer arrays
 13. Binary search + rolling hash
 14. KMP vs Rabin-Karp vs Trie
-

1. What Is Rolling Hash?

A hash converts a string into a number.

Example:

```
"abc" -> 96354
"bca" -> 97344
```

The exact number is not important.

The goal is:

```
If two strings are equal, their hashes should be equal.
If hashes are different, strings are definitely different.
If hashes are equal, strings are probably equal.
```

Rolling hash lets us compute hashes of substrings quickly.

Instead of recomputing hash from scratch for every substring, we precompute prefix hashes.

Then any substring hash can be obtained in $O(1)$.

2. Why Rolling Hash Exists

Suppose you need to compare many substrings.

Example:

```
s = "banana"
```

Compare:

```
s[1..3] = "ana"
s[3..5] = "ana"
```

Direct comparison costs $O(\text{length})$.

If there are many comparisons, this becomes expensive.

Rolling hash allows:

```
hash(1, 3) == hash(3, 5)
```

in $O(1)$.

This helps with:

- duplicate substring detection
 - repeated pattern detection
 - palindrome checking
 - binary search on answer length
 - distinct substring counting
-

3. Polynomial Rolling Hash

A common string hash is polynomial hash.

For string:

```
s = "abc"
```

Represent characters as numbers:

```
a = 1
b = 2
c = 3
```

Hash:

```
hash("abc") = 1 * base2 + 2 * base1 + 3 * base0
```

or equivalently when scanning left to right:

```
hash = hash * base + value
```

Example:

```
h = 0
h = h * base + 1
h = h * base + 2
h = h * base + 3
```

4. Modulo Hashing

Hash values grow very large, so use modulo.

```
MOD = 1,000,000,007
```

or another large prime.

Update:

```
hash = (hash * BASE + value) % MOD;
```

However, modulo introduces collision risk:

```
different strings may have same hash
```

To reduce risk, use double hashing.

5. Important Collision Warning

Rolling hash is probabilistic if using modulo.

Two different strings can theoretically have same hash.

This is called a collision.

In interviews:

- mention collision risk
- use double hashing if needed
- verify actual substring if required

Double hashing uses two different mod values.

Example:

```
hash = (hash1, hash2)
```

Collision probability becomes extremely small.

6. Prefix Hash

For string s, define:

```
prefix[i] = hash of s[0..i-1]
```

So:

```
prefix[0] = 0  
prefix[1] = hash of first char  
prefix[2] = hash of first 2 chars
```

Build:

```
prefix[i + 1] = prefix[i] * BASE + value(s[i])
```

Also precompute powers:

```
power[i] = BASE^i
```

7. Substring Hash Formula

To get hash of:

```
s[l..r-1]
```

length:

```
len = r - l
```

Formula:

```
hash = prefix[r] - prefix[l] * power[len]
```

With modulo:

```
hash = (prefix[r] - prefix[l] * power[len] % MOD + MOD) % MOD;
```

This gives $O(1)$ substring hash.

8. Rolling Hash Template

```
class RollingHash {  
public:  
    static const long long MOD = 1000000007;  
    static const long long BASE = 911382323;  
  
    vector<long long> prefix;  
    vector<long long> power;  
  
    RollingHash(string s) {  
        int n = s.size();  
  
        prefix.assign(n + 1, 0);  
        power.assign(n + 1, 1);  
  
        for (int i = 0; i < n; i++) {  
            int value = s[i];  
  
            prefix[i + 1] = (prefix[i] * BASE + value) % MOD;
```

```

        power[i + 1] = (power[i] * BASE) % MOD;
    }
}

long long getHash(int l, int r) {
    long long result = prefix[r] - prefix[l] * power[r - l] %
MOD;

    if (result < 0) {
        result += MOD;
    }

    return result;
}
};

```

Note About BASE

In real use, choose base smaller than MOD and preferably random-ish.
Common choices:

```

31
53
911382323

```

If using huge base, ensure multiplication uses long long.

9. Safer Double Hash Template

```

class DoubleHash {
public:
    static const long long MOD1 = 1000000007;
    static const long long MOD2 = 1000000009;
    static const long long BASE = 911382323;

    vector<long long> h1;
    vector<long long> h2;
    vector<long long> p1;
    vector<long long> p2;

    DoubleHash(string s) {
        int n = s.size();

        h1.assign(n + 1, 0);
        h2.assign(n + 1, 0);
        p1.assign(n + 1, 1);
        p2.assign(n + 1, 1);

        for (int i = 0; i < n; i++) {
            int value = s[i];

```

```

        h1[i + 1] = (h1[i] * BASE + value) % MOD1;
        h2[i + 1] = (h2[i] * BASE + value) % MOD2;

        p1[i + 1] = (p1[i] * BASE) % MOD1;
        p2[i + 1] = (p2[i] * BASE) % MOD2;
    }
}

pair<long long, long long> getHash(int l, int r) {
    long long x1 = h1[r] - h1[l] * p1[r - l] % MOD1;
    if (x1 < 0) {
        x1 += MOD1;
    }

    long long x2 = h2[r] - h2[l] * p2[r - l] % MOD2;
    if (x2 < 0) {
        x2 += MOD2;
    }

    return {x1, x2};
}
};

```

10. When to Use Rolling Hash

Use rolling hash when:

10.1 Many Substring Comparisons

Example:

Compare $s[l1..r1]$ and $s[l2..r2]$ many times.

10.2 Duplicate Substring Detection

Example:

Longest Duplicate Substring

10.3 Binary Search on Substring Length

Check if duplicate substring of length L exists.

10.4 Count Distinct Substrings

Hash all substrings.

10.5 Palindrome Checking

Compare forward hash with reverse hash.

10.6 Pattern Matching

Rabin-Karp.

11. When Not to Use Rolling Hash

Do not use rolling hash when:

11.1 Exact Deterministic Matching Is Required

KMP is deterministic and collision-free.

11.2 Prefix Search Is Needed

Use Trie.

11.3 Edit/Subsequence Optimization Is Needed

Use DP.

11.4 Simple Problem Is Solvable by Hash Set

Do not overcomplicate.

11.5 Collision Risk Is Unacceptable

Use deterministic algorithms or verify matches.

12. KMP vs Rabin-Karp vs Trie

Technique	Best For	Weakness
KMP	exact single-pattern search	less flexible for arbitrary substring comparison
Rabin-Karp	substring hashing, multiple comparisons	collision risk
Trie	prefix/dictionary search	memory-heavy, prefix-focused
Rolling Hash	duplicate substring, palindrome checks	collision risk
DP	edit/subsequence/palindrome subsequence	often $O(n^2)$

13. Pattern Recognition Checklist

Ask:

15. Do I need to compare many substrings?
 16. Do I need duplicate substring detection?
 17. Is there binary search on substring length?
 18. Do I need all substrings of fixed length?
 19. Is this pattern matching?
 20. Can KMP solve it more safely?
 21. Is prefix search involved, suggesting Trie?
 22. Can I tolerate collision risk?
 23. Should I use double hashing?
 24. Do I need to verify substrings after hash match?
-

14. Full Problem Walkthrough 1 — Rabin-Karp Pattern Matching

14.1 Problem Summary

Find the first occurrence of pattern in text.

Example:

```
text = "hello"
pattern = "ll"
```

Answer:

2

14.2 Idea

Compute hash of pattern.

Compute hash of every text substring of same length.

If hash matches, verify substring to avoid collision.

14.3 C++ Code

```
class Solution {
public:
    static const long long MOD = 1000000007;
    static const long long BASE = 911382323;

    int rabinKarp(string text, string pattern) {
        int n = text.size();
        int m = pattern.size();

        if (m > n) {
            return -1;
        }

        vector<long long> power(n + 1, 1);
        vector<long long> prefix(n + 1, 0);

        for (int i = 0; i < n; i++) {
            power[i + 1] = power[i] * BASE % MOD;
            prefix[i + 1] = (prefix[i] * BASE + text[i]) % MOD;
        }

        long long patternHash = 0;

        for (char c : pattern) {
            patternHash = (patternHash * BASE + c) % MOD;
        }

        auto getHash = [&](int l, int r) {
            long long h = prefix[r] - prefix[l] * power[r - l] %
```

```

MOD;

    if (h < 0) {
        h += MOD;
    }

    return h;
};

for (int i = 0; i + m <= n; i++) {
    if (getHash(i, i + m) == patternHash) {
        if (text.substr(i, m) == pattern) {
            return i;
        }
    }
}

return -1;
}
};

```

14.4 Complexity

Expected:

$O(n + m)$

Worst-case with many collisions and verification:

$O(nm)$

Double hashing reduces collision risk.

15. Full Problem Walkthrough 2 — LeetCode 187: Repeated DNA Sequences

15.1 Problem Summary

Given DNA string containing:

A, C, G, T

Find all 10-letter-long sequences that occur more than once.

15.2 Approach

Since length is fixed at 10, use hash set.

Could use string directly or rolling hash.

We show simple encoding because DNA has only 4 chars.

15.3 2-bit Encoding

Map:

A -> 0
C -> 1
G -> 2
T -> 3

Each char uses 2 bits.

10 chars use 20 bits.

15.4 C++ Code

```
class Solution {
public:
    int code(char c) {
        if (c == 'A') {
            return 0;
        }

        if (c == 'C') {
            return 1;
        }

        if (c == 'G') {
            return 2;
        }

        return 3;
    }

    vector<string> findRepeatedDnaSequences(string s) {
        vector<string> result;

        if (s.size() < 10) {
            return result;
        }

        unordered_set<int> seen;
        unordered_set<int> repeated;

        int hash = 0;
        int mask = (1 << 20) - 1;

        for (int i = 0; i < s.size(); i++) {
            hash = ((hash << 2) | code(s[i])) & mask;

            if (i >= 9) {
                if (seen.count(hash) && !repeated.count(hash)) {
                    result.push_back(s.substr(i - 9, 10));
                    repeated.insert(hash);
                }

                seen.insert(hash);
            }
        }
    }
};
```

```

        }
    }

    return result;
}
};

```

15.5 Complexity

Time:

$O(n)$

Space:

$O(n)$

16. Full Problem Walkthrough 3 — Count Distinct Substrings

16.1 Problem Summary

Count number of distinct substrings of s .

Example:

```
s = "ababa"
```

Distinct substrings include:

a, b, ab, ba, aba, bab, ...

16.2 Rolling Hash Approach

For every length L :

- compute hash of every substring of length L
- add to set

16.3 C++ Code

```

class Solution {
public:
    using ll = long long;

    static const ll MOD1 = 1000000007;
    static const ll MOD2 = 1000000009;
    static const ll BASE = 911382323;

    int countDistinctSubstrings(string s) {
        int n = s.size();

        vector<ll> h1(n + 1, 0), h2(n + 1, 0);
        vector<ll> p1(n + 1, 1), p2(n + 1, 1);

        for (int i = 0; i < n; i++) {
            int value = s[i];

```

```

        h1[i + 1] = (h1[i] * BASE + value) % MOD1;
        h2[i + 1] = (h2[i] * BASE + value) % MOD2;

        p1[i + 1] = p1[i] * BASE % MOD1;
        p2[i + 1] = p2[i] * BASE % MOD2;
    }

    auto getHash = [&](int l, int r) {
        ll x1 = h1[r] - h1[l] * p1[r - l] % MOD1;
        if (x1 < 0) {
            x1 += MOD1;
        }

        ll x2 = h2[r] - h2[l] * p2[r - l] % MOD2;
        if (x2 < 0) {
            x2 += MOD2;
        }

        return pair<ll, ll>{x1, x2};
    };

    set<pair<ll, ll>> all;

    for (int len = 1; len <= n; len++) {
        for (int l = 0; l + len <= n; l++) {
            all.insert(getHash(l, l + len));
        }
    }

    return all.size();
}
};

```

16.4 Complexity

Time:

$O(n^2 \log n)$

Space:

$O(n^2)$

16.5 Note

Suffix automaton can count distinct substrings in $O(n)$, but it is more advanced.

17. Full Problem Walkthrough 4 — LeetCode 1044: Longest Duplicate Substring

17.1 Problem Summary

Given string s , return any longest substring that appears at least twice.

Example:

```
s = "banana"
```

Answer:

```
"ana"
```

17.2 Why Rolling Hash?

Need check whether duplicate substring of length L exists.

Then binary search L .

Check(L):

- compute all hashes of substrings of length L
- if any hash repeats, duplicate may exist

17.3 Binary Search

If duplicate of length L exists, duplicate of smaller length also exists.

So use binary search on length.

17.4 C++ Code With Double Hash

```
class Solution {
public:
    using ll = long long;

    static const ll MOD1 = 1000000007;
    static const ll MOD2 = 1000000009;
    static const ll BASE = 911382323;

    string longestDupSubstring(string s) {
        int n = s.size();

        vector<ll> h1(n + 1, 0), h2(n + 1, 0);
        vector<ll> p1(n + 1, 1), p2(n + 1, 1);

        for (int i = 0; i < n; i++) {
            int value = s[i];

            h1[i + 1] = (h1[i] * BASE + value) % MOD1;
            h2[i + 1] = (h2[i] * BASE + value) % MOD2;

            p1[i + 1] = p1[i] * BASE % MOD1;
            p2[i + 1] = p2[i] * BASE % MOD2;
        }
    }
};
```

```

auto getHash = [&](int l, int r) {
    ll x1 = h1[r] - h1[l] * p1[r - l] % MOD1;
    if (x1 < 0) {
        x1 += MOD1;
    }

    ll x2 = h2[r] - h2[l] * p2[r - l] % MOD2;
    if (x2 < 0) {
        x2 += MOD2;
    }

    return pair<ll, ll>{x1, x2};
};

auto check = [&](int len) {
    unordered_map<long long, vector<int>> seen;

    for (int i = 0; i + len <= n; i++) {
        auto [a, b] = getHash(i, i + len);

        long long key = (a << 32) ^ b;

        if (seen.count(key)) {
            for (int start : seen[key]) {
                if (s.compare(start, len, s, i, len) == 0) {
                    return i;
                }
            }
        }

        seen[key].push_back(i);
    }

    return -1;
};

int left = 1;
int right = n - 1;

int bestStart = -1;
int bestLen = 0;

while (left <= right) {
    int mid = left + (right - left) / 2;

    int start = check(mid);

    if (start != -1) {

```

```

        bestStart = start;
        bestLen = mid;
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}

if (bestStart == -1) {
    return "";
}

return s.substr(bestStart, bestLen);
}
};

```

17.5 Important Notes

We used double hash plus actual comparison on collision.
This is safe.

17.6 Complexity

Expected:

$O(n \log n)$

Verification can increase time in collision-heavy cases, but double hashing keeps it practical.

18. Key Implementation Warning — Combining Hash Pair

In code above:

```
long long key = (a << 32) ^ b;
```

This assumes values fit safely.

Alternative safer approach:

Use `set<pair<ll, ll>>` or custom hash for pair.

Example custom hash:

```

struct PairHash {
    size_t operator()(const pair<long long, long long>& p) const {
        return hash<long long>()(p.first ^ (p.second << 1));
    }
};

```

Then:

```
unordered_map<pair<ll, ll>, vector<int>, PairHash> seen;
```

This avoids bit-packing concerns.

19. Full Problem Walkthrough 5 —

LeetCode 1316: Distinct Echo Substrings

19.1 Problem Summary

An echo substring is a string that can be written as:

$a + a$

Example:

"abcabc"

where first half equals second half.

Return number of distinct echo substrings.

19.2 Rolling Hash Idea

For every possible length len of half:

Check substrings:

$s[i..i+len-1]$
 $s[i+len..i+2len-1]$

If hashes equal, add hash of full substring to set.

19.3 C++ Code

```
class Solution {
public:
    using ll = long long;

    static const ll MOD1 = 1000000007;
    static const ll MOD2 = 1000000009;
    static const ll BASE = 911382323;

    int distinctEchoSubstrings(string text) {
        int n = text.size();

        vector<ll> h1(n + 1, 0), h2(n + 1, 0);
        vector<ll> p1(n + 1, 1), p2(n + 1, 1);

        for (int i = 0; i < n; i++) {
            int value = text[i];

            h1[i + 1] = (h1[i] * BASE + value) % MOD1;
            h2[i + 1] = (h2[i] * BASE + value) % MOD2;

            p1[i + 1] = p1[i] * BASE % MOD1;
            p2[i + 1] = p2[i] * BASE % MOD2;
        }
    }
};
```

```

auto getHash = [&](int l, int r) {
    ll x1 = h1[r] - h1[l] * p1[r - l] % MOD1;
    if (x1 < 0) {
        x1 += MOD1;
    }

    ll x2 = h2[r] - h2[l] * p2[r - l] % MOD2;
    if (x2 < 0) {
        x2 += MOD2;
    }

    return pair<ll, ll>{x1, x2};
};

set<pair<ll, ll>> distinct;

for (int len = 1; 2 * len <= n; len++) {
    for (int i = 0; i + 2 * len <= n; i++) {
        auto first = getHash(i, i + len);
        auto second = getHash(i + len, i + 2 * len);

        if (first == second) {
            distinct.insert(getHash(i, i + 2 * len));
        }
    }
}

return distinct.size();
};

```

19.4 Complexity

Time:

$O(n^2 \log n)$

Space:

$O(n^2)$

20. Full Problem Walkthrough 6 — Longest Happy Prefix With Hash

20.1 Problem Summary

Find longest prefix that is also suffix, excluding the full string.

Example:

`s = "level"`

Answer:

"1"

20.2 KMP vs Hash

KMP is deterministic and preferred.

Hash solution is simpler conceptually:

Compare hash of prefix length L with suffix length L.

20.3 C++ Code

```
class Solution {
public:
    string longestPrefix(string s) {
        const long long MOD = 1000000007;
        const long long BASE = 911382323;

        int n = s.size();

        long long prefixHash = 0;
        long long suffixHash = 0;
        long long power = 1;

        int best = 0;

        for (int len = 1; len < n; len++) {
            prefixHash = (prefixHash * BASE + s[len - 1]) % MOD;

            suffixHash = (suffixHash + power * s[n - len]) % MOD;
            power = power * BASE % MOD;

            if (prefixHash == suffixHash) {
                best = len;
            }
        }

        return s.substr(0, best);
    }
};
```

20.4 Note

This uses one hash. Double hash is safer.

21. Full Problem Walkthrough 7 — Shortest Palindrome With Rolling Hash

21.1 Problem Summary

Add characters in front to make shortest palindrome.

Need longest palindromic prefix.

21.2 Hash Idea

Scan prefix and maintain:

- forward hash
- reverse hash

When they match, prefix might be palindrome.

21.3 C++ Code

```
class Solution {
public:
    string shortestPalindrome(string s) {
        const long long MOD = 1000000007;
        const long long BASE = 911382323;

        long long forward = 0;
        long long backward = 0;
        long long power = 1;

        int best = 0;

        for (int i = 0; i < s.size(); i++) {
            int value = s[i];

            forward = (forward * BASE + value) % MOD;
            backward = (backward + power * value) % MOD;
            power = power * BASE % MOD;

            if (forward == backward) {
                best = i + 1;
            }
        }

        string suffix = s.substr(best);
        reverse(suffix.begin(), suffix.end());

        return suffix + s;
    }
};
```

21.4 Collision Warning

KMP version is deterministic.

Hash version is elegant but probabilistic.

22. Full Problem Walkthrough 8 — Palindrome Substring Query With Hash

22.1 Problem Summary

Given string s and many queries $[l, r]$, check if $s[l..r]$ is palindrome.

Naive per query:

$O(\text{length})$

Hash method:

$O(1)$

22.2 Idea

Build forward hash for s .

Build forward hash for reversed s .

Substring $s[l..r]$ corresponds to reversed substring in rev :

$rev[n - 1 - r .. n - 1 - l]$

Compare hashes.

22.3 C++ Code

```
class PalindromeHash {
public:
    using ll = long long;

    static const ll MOD = 1000000007;
    static const ll BASE = 911382323;

    string s;
    string rev;
    vector<ll> h1;
    vector<ll> h2;
    vector<ll> power;
    int n;

    PalindromeHash(string str) {
        s = str;
        rev = s;
        reverse(rev.begin(), rev.end());

        n = s.size();

        h1.assign(n + 1, 0);
        h2.assign(n + 1, 0);
        power.assign(n + 1, 1);
    }
};
```

```

        for (int i = 0; i < n; i++) {
            h1[i + 1] = (h1[i] * BASE + s[i]) % MOD;
            h2[i + 1] = (h2[i] * BASE + rev[i]) % MOD;
            power[i + 1] = power[i] * BASE % MOD;
        }
    }

    ll getHash(vector<ll>& h, int l, int r) {
        ll result = h[r] - h[l] * power[r - l] % MOD;

        if (result < 0) {
            result += MOD;
        }

        return result;
    }

    bool isPalindrome(int l, int r) {
        int revL = n - 1 - r;
        int revR = n - 1 - l;

        return getHash(h1, l, r + 1) == getHash(h2, revL, revR);
    }
};

```

23. Full Problem Walkthrough 9 — Rolling Hash for Integer Arrays

Rolling hash is not only for strings.

You can hash integer arrays too.

Example:

Find repeated subarrays of length k.

Use:

```
hash = hash * BASE + nums[i]
```

But if `nums[i]` can be large or negative, normalize:

```
value = nums[i] + OFFSET
```

Useful for:

- repeated subarray patterns
- sequence matching
- array pattern matching

24. Full Problem Walkthrough 10 — Longest Common Subpath Preview

This is a hard rolling-hash problem.

Given multiple paths, find longest common subpath among all.

Pattern:

- binary search length L
- compute rolling hashes of all length-L subpaths for each path
- intersect sets across paths
- if intersection non-empty, length L exists

This combines:

binary search + rolling hash + set intersection

Important for advanced practice.

25. Hash Collision Handling

25.1 Ways to Reduce Risk

Use Double Hash

(hash1, hash2)

Use Large Random Base

Avoid tiny bases for adversarial input.

Verify Actual Strings

When hash match found, compare actual substring.

Use unsigned long long Hash

Some use natural overflow modulo 2^{64} .

Fast but still probabilistic.

25.2 Interview Explanation

Say:

Rolling hash can have collisions, so for safety I can use double hashing or verify substrings when hashes match.

This shows awareness.

26. Common Rolling Hash Edge Cases

26.1 Empty String

Hash is 0.

26.2 Pattern Longer Than Text

Return false / -1.

26.3 Negative Modulo

Always fix:

```
if (hash < 0) {  
    hash += MOD;  
}
```

26.4 Character Value 0

Avoid mapping characters to 0 if using simple polynomial hash.

Using ASCII value is fine.

26.5 Large Multiplication

Use long long.

If MOD and BASE are very large, multiplication may overflow. Use safer multiplication if needed.

26.6 Collision

Remember collision risk.

27. Common Rolling Hash Mistakes

Mistake 1 — Wrong Substring Formula

Correct:

```
prefix[r] - prefix[l] * power[r - l]
```

Mistake 2 — Forgetting Mod Correction

Subtraction can be negative.

Mistake 3 — Not Precomputing Powers

Need powers for $O(1)$ substring hash.

Mistake 4 — Comparing Hashes of Different Lengths

Normally only compare same-length substrings.

Mistake 5 — Using One Hash in Adversarial Settings

Use double hash.

Mistake 6 — Off-by-One r Exclusive vs Inclusive

Be consistent.

Recommended:

```
getHash(l, r) means [l, r)
```

28. Interview Explanation Templates

28.1 Rolling Hash

I precompute prefix hashes and powers of the base. Then I can get the hash of any substring in $O(1)$ using the prefix hash formula.

28.2 Rabin-Karp

I compute the hash of the pattern and compare it with rolling hashes of text substrings of the same length. On a hash match, I can verify the substring to avoid collision issues.

28.3 Longest Duplicate Substring

I binary search the answer length. For a fixed length, I use rolling hash to check whether any substring of that length appears more than once.

28.4 Collision Handling

Hash collisions are possible, so I use double hashing or verify actual substrings when hash values match.

28.5 Palindrome Query

I compare the forward hash of the substring with the corresponding hash in the reversed string.

29. Complexity Patterns

Problem	Time	Space
Build prefix hash	$O(n)$	$O(n)$
Get substring hash	$O(1)$	$O(1)$
Rabin-Karp	$O(n + m)$ expected	$O(n)$
Repeated DNA	$O(n)$	$O(n)$
Count distinct substrings	$O(n^2 \log n)$	$O(n^2)$
Longest duplicate substring	$O(n \log n)$ expected	$O(n)$
Distinct echo substrings	$O(n^2 \log n)$	$O(n^2)$
Palindrome queries	$O(n + q)$	$O(n)$
Longest common subpath	$O(\text{total length} * \log \text{minLen})$ expected	$O(\text{total length})$

30. Expanded Practice Set

30.1 Must-Do Medium

- 25. Rabin-Karp Pattern Matching
- 26. Repeated DNA Sequences
- 27. Longest Happy Prefix with hash
- 28. Shortest Palindrome with hash
- 29. Palindrome substring queries
- 30. Count distinct substrings with hash

30.2 Strong Interview Practice

- 31. Longest Duplicate Substring
- 32. Distinct Echo Substrings
- 33. Repeated String Match with Rabin-Karp
- 34. Find duplicate subarrays
- 35. Binary search + rolling hash practice
- 36. Double hash implementation

30.3 Hard / Stretch

- 37. Longest Common Subpath
 - 38. Palindrome Pairs with hash
 - 39. Count distinct palindromic substrings with hash
 - 40. Suffix Array alternative for Longest Duplicate Substring
 - 41. Suffix Automaton alternative for distinct substrings
-

31. Suggested Study Order

Step 1 — Hash Basics

Solve:

- 42. Rabin-Karp
- 43. Repeated DNA Sequences

Goal:

Understand rolling update and hash matching.

Step 2 — Prefix Hash

Solve:

- 44. Substring hash queries
- 45. Palindrome substring queries

Goal:

Master $O(1)$ substring hash.

Step 3 — Binary Search + Hash

Solve:

- 46. Longest Duplicate Substring

Goal:

Learn `check(length)` pattern.

Step 4 — Distinct Substrings

Solve:

- 47. Count distinct substrings
- 48. Distinct Echo Substrings

Goal:

Use sets of hashes.

Step 5 — Advanced

Study:

- 49. Longest Common Subpath
- 50. Suffix array alternative
- 51. Suffix automaton alternative

Goal:

Understand limits of rolling hash.

32. Pattern Decision Table

Problem Clue	Rolling Hash Strategy
find pattern in text	Rabin-Karp
compare many substrings	prefix hash
duplicate substring exists	hash set of fixed length
longest duplicate substring	binary search + hash
count distinct substrings	set of substring hashes

Problem Clue	Rolling Hash Strategy
repeated DNA length 10	rolling encoded hash
palindrome substring query	forward hash vs reverse hash
prefix equals suffix	compare prefix/suffix hash
echo substring	compare adjacent equal-length hashes
common subpath	binary search + hash set intersection

33. Mini Problem 1 — Fixed-Length Duplicate Substring

Given s and length L , return true if any substring of length L repeats.

```
bool hasDuplicateLength(string s, int len) {
    DoubleHash dh(s);

    set<pair<long long, long long>> seen;

    for (int i = 0; i + len <= s.size(); i++) {
        auto h = dh.getHash(i, i + len);

        if (seen.count(h)) {
            return true;
        }

        seen.insert(h);
    }

    return false;
}
```

34. Mini Problem 2 — Compare Two Substrings

Check if:

```
s[l1..r1] == s[l2..r2]
```

Use same length and hash.

```
bool equalSubstring(DoubleHash& dh, int l1, int r1, int l2, int r2)
{
    if (r1 - l1 != r2 - l2) {
        return false;
    }
```

```

    }

    return dh.getHash(l1, r1) == dh.getHash(l2, r2);
}

```

35. Mini Problem 3 — Count Distinct Substrings of Fixed Length

```

int countDistinctLength(string s, int len) {
    DoubleHash dh(s);

    set<pair<long long, long long>> st;

    for (int i = 0; i + len <= s.size(); i++) {
        st.insert(dh.getHash(i, i + len));
    }

    return st.size();
}

```

36. Mini Problem 4 — Pattern Occurrences With Hash

```

vector<int> findOccurrences(string text, string pattern) {
    vector<int> result;

    if (pattern.size() > text.size()) {
        return result;
    }

    DoubleHash textHash(text);
    DoubleHash patternHash(pattern);

    auto target = patternHash.getHash(0, pattern.size());

    int m = pattern.size();

    for (int i = 0; i + m <= text.size(); i++) {
        if (textHash.getHash(i, i + m) == target) {
            if (text.compare(i, m, pattern) == 0) {
                result.push_back(i);
            }
        }
    }
}

```

```
    return result;  
}
```

37. Common Edge Case Checklist

Before submitting rolling hash code, test:

```
empty string  
one character  
pattern longer than text  
all same characters  
no duplicate substring  
duplicate at beginning  
duplicate at end  
very long string  
hash subtraction negative  
same hash length comparison  
palindrome odd length  
palindrome even length
```

38. Debugging Rolling Hash

If rolling hash fails, check:

- 52. Is `getHash` using `[l, r)` consistently?
- 53. Are powers precomputed correctly?
- 54. Is prefix hash built correctly?
- 55. Did you fix negative modulo?
- 56. Are you comparing substrings of same length?
- 57. Is `BASE` smaller than `MOD` if using modular multiplication?
- 58. Could collision be causing issue?
- 59. Should you use double hashing?
- 60. Are you verifying actual substring on hash match?
- 61. Is binary search updating answer correctly?

39. What to Master Before Chapter 48

Before moving to Parsing, make sure you can:

- Explain rolling hash
- Build prefix hash
- Get substring hash in $O(1)$
- Explain collision risk
- Use double hashing
- Implement Rabin-Karp
- Solve Repeated DNA Sequences
- Solve Longest Duplicate Substring conceptually
- Use binary search + hash
- Use hash for palindrome substring queries

- Know when KMP is better than rolling hash
 - Know when Trie is better than rolling hash
-

40. Final Cheat Sheet

Prefix Hash Build

```
prefix[i + 1] = (prefix[i] * BASE + value) % MOD;  
power[i + 1] = power[i] * BASE % MOD;
```

Substring Hash

```
hash = prefix[r] - prefix[l] * power[r - l] % MOD;  
  
if (hash < 0) {  
    hash += MOD;  
}
```

Rabin-Karp

```
if (textHash.getHash(i, i + m) == patternHash) {  
    verify substring;  
}
```

Duplicate Fixed Length

```
if hash already seen:  
    duplicate exists
```

Binary Search + Hash

```
if duplicate length mid exists:  
    try larger  
else:  
    try smaller
```

Palindrome Query

```
hash in original == corresponding hash in reversed string
```

Collision Safety

```
Use double hash or verify actual substring.
```

41. Core Takeaway

Rolling hash is a tool for fast substring comparison.
The main idea is:

```
Precompute prefix hashes.  
Then compare any substring in  $O(1)$ .
```

It is especially useful when combined with:

- hash sets
- binary search

- palindrome checks
- duplicate detection

Rabin-Karp is the pattern-matching use case of rolling hash.

The main danger is collision.

For interview-quality code, mention:

I can use double hashing or verify actual substrings to avoid collision issues.

Choose the right tool:

- KMP for deterministic single-pattern search
 - Trie for prefix dictionaries
 - Rolling hash for many substring comparisons
 - DP for edit/subsequence optimization
-